Nama : Serlin Aprilia

NIM : 109082530032

Kelas : S1IF-13-06

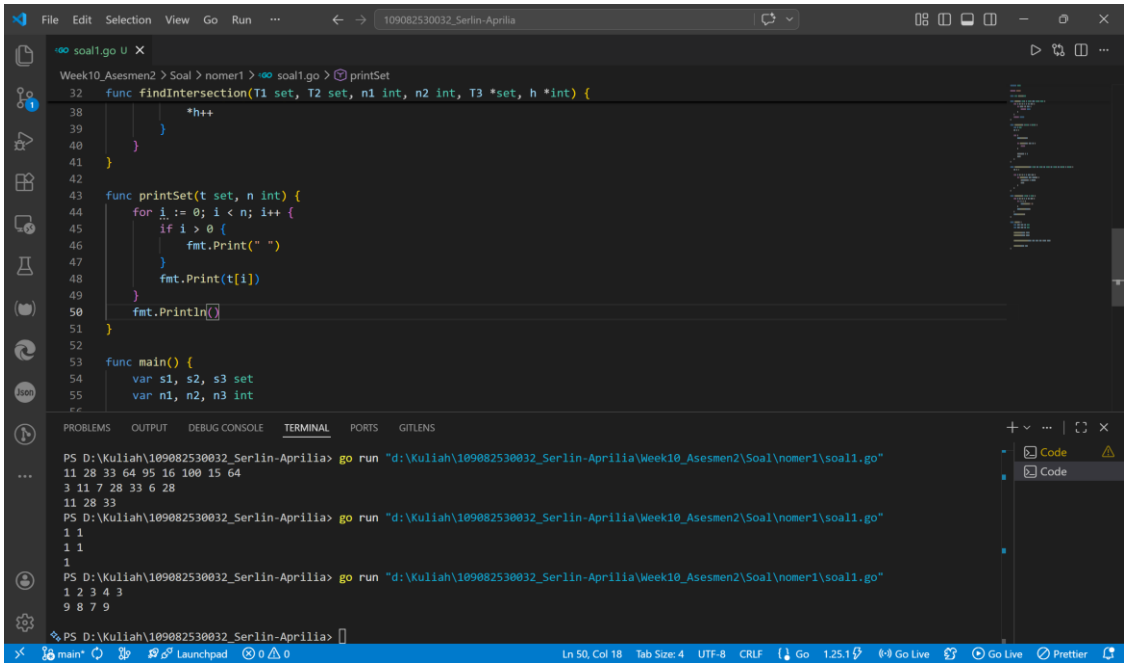
JAWABAN ASESMEN 2 TIPE DATA TERSTRUKTUR

1. Source Code jawaban :

```go
package main

import "fmt"

type set [2022]int

func exist(t set, n int, val int) bool {
    for i := 0; i < n; i++ {
        if t[i] == val {
            return true
        }
    }
    return false
}

func inputSet(t *set, n *int) {
    var x int
    *n = 0

    for {
        fmt.Scan(&x)

        if exist(*t, *n, x) {
            break
        }

        (*t)[*n] = x
        *n++
    }
}

func findIntersection(T1 set, T2 set, n1 int, n2 int, T3 *set, h *int) {
```

```go
    *h = 0

    for i := 0; i < n1; i++ {
        if exist(T2, n2, T1[i]) {
            (*T3)[*h] = T1[i]
            *h++
        }
    }
}

func printSet(t set, n int) {
    for i := 0; i < n; i++ {
        if i > 0 {
            fmt.Print(" ")
        }
        fmt.Print(t[i])
    }
    fmt.Println()
}

func main() {
    var s1, s2, s3 set
    var n1, n2, n3 int

    inputSet(&s1, &n1)
    inputSet(&s2, &n2)

    findIntersection(s1, s2, n1, n2, &s3, &n3)

    printSet(s3, n3)
}
```

Screenshot Output :



Penjelasan Singkat Cara Kerja Program :

Program dimulai dari fungsi main dengan mendeklarasikan tiga variabel bertipe set yaitu s1, s2, dan s3, serta tiga variabel integer n1, n2, dan n3 sebagai penanda jumlah elemen masing-masing himpunan. Selanjutnya, fungsi inputSet dipanggil dua kali untuk mengisi s1 dan s2. Di dalam inputSet, variabel jumlah elemen diinisialisasi ke 0, kemudian program membaca input menggunakan fmt.Scan secara berulang (loop tak hingga). Setiap nilai yang dibaca akan dicek menggunakan fungsi exist, yaitu dengan melakukan iterasi dari indeks 0 sampai n-1 untuk membandingkan apakah nilai tersebut sudah ada di array. Jika ditemukan nilai yang sama, loop dihentikan (break), jika tidak, nilai disimpan pada indeks ke-n dan n ditambahkan satu. Setelah kedua set terbentuk, fungsi findIntersection dipanggil dengan parameter kedua set dan jumlah elemennya. Di dalam fungsi ini, variabel hasil h diinisialisasi ke 0, lalu dilakukan perulangan pada seluruh elemen T1. Untuk setiap elemen T1[i], fungsi exist dipanggil kembali untuk mengecek keberadaannya di T2. Jika ditemukan, maka nilai tersebut disimpan ke array hasil T3 pada indeks h, kemudian h ditambah satu. Setelah proses selesai, T3 berisi semua elemen yang merupakan irisan dari kedua set. Terakhir, fungsi printSet dipanggil untuk mencetak isi T3 sebanyak n3 elemen dengan format dipisahkan spasi, menggunakan loop dari indeks 0 hingga n3-1. Secara teknis, alur data mengalir dari input → validasi duplikat → penyimpanan array → pencarian irisan → output hasil.

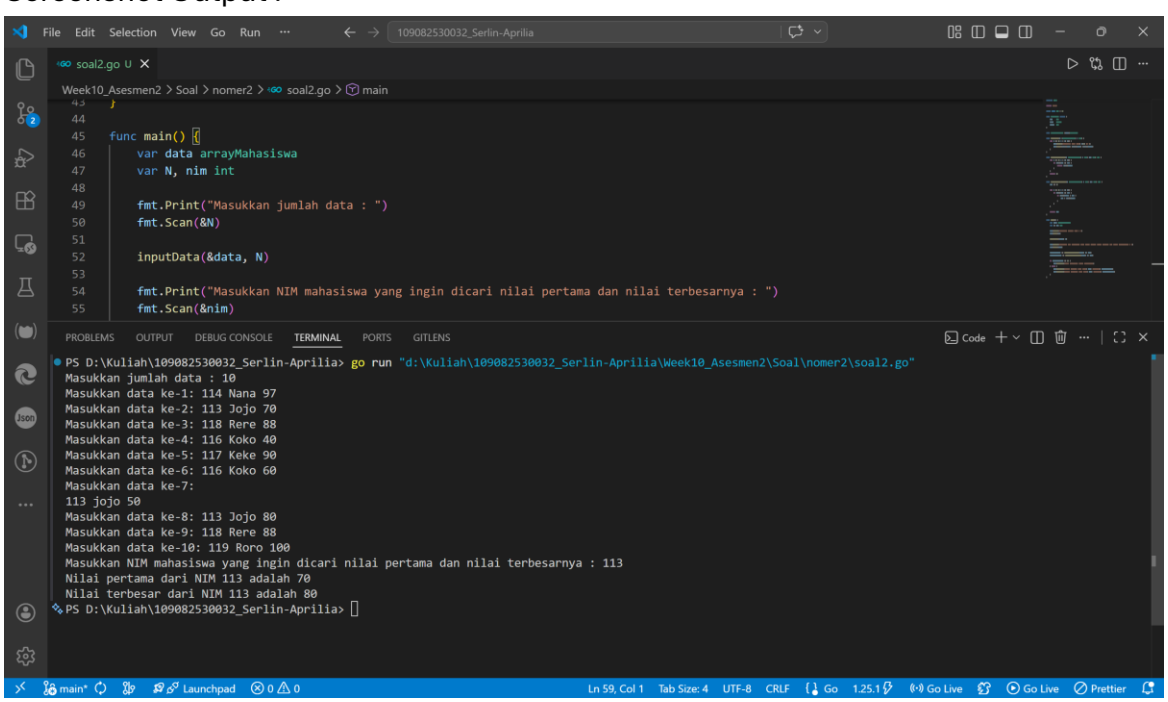
2. Sourec Code Jawaban :

```
3. package main
```

```go

import "fmt"

const nMax int = 51

type mahasiswa struct {
    NIM    int
    nama   string
    nilai  int
}

type arrayMahasiswa [nMax]mahasiswa

func inputData(T *arrayMahasiswa, N int) {
    for i := 0; i < N; i++ {
        fmt.Print("Masukkan data ke-", i+1, ": ")
        fmt.Scan(&T[i].NIM, &T[i].nama, &T[i].nilai)
    }
}

func cariNilaiPertama(T arrayMahasiswa, N int, nim int) int {
    for i := 0; i < N; i++ {
        if T[i].NIM == nim {
            return T[i].nilai
        }
    }
    return -1
}

func cariNilaiTerbesar(T arrayMahasiswa, N int, nim int) int {
    max := -1

    for i := 0; i < N; i++ {
        if T[i].NIM == nim {
            if T[i].nilai > max {
                max = T[i].nilai
            }
        }
    }

    return max
}

func main() {
    var data arrayMahasiswa
```

```go
    var N, nim int

    fmt.Print("Masukkan jumlah data : ")
    fmt.Scan(&N)

    inputData(&data, N)

    fmt.Print("Masukkan NIM mahasiswa yang ingin dicari nilai pertama dan nilai terbesarnya : ")
    fmt.Scan(&nim)

    nilaiPertama := cariNilaiPertama(data, N, nim)
    nilaiTerbesar := cariNilaiTerbesar(data, N, nim)

    if nilaiPertama == -1 {
        fmt.Println("Data mahasiswa tidak ditemukan")
    } else {
        fmt.Println("Nilai pertama dari NIM", nim, "adalah", nilaiPertama)
        fmt.Println("Nilai terbesar dari NIM", nim, "adalah", nilaiTerbesar)
    }
}
```

Screenshot Output :



Penjelasan Singkat Cara Kerja Program :

Program dimulai dari fungsi main dengan mendeklarasikan variabel data bertipe arrayMahasiswa yang digunakan untuk menyimpan kumpulan data mahasiswa, serta variabel N sebagai jumlah data dan nim sebagai nilai NIM yang akan dicari. Program kemudian meminta input jumlah data (N) menggunakan fmt.Scan, lalu memanggil fungsi inputData untuk mengisi array. Di dalam inputData, dilakukan perulangan dari indeks 0 sampai N-1, dan pada setiap iterasi program membaca tiga atribut mahasiswa yaitu NIM, nama, dan nilai, lalu menyimpannya ke dalam array pada indeks yang sesuai.

Setelah seluruh data tersimpan, program meminta input NIM yang ingin dicari. Selanjutnya dilakukan dua proses pencarian. Pertama, fungsi cariNilaiPertama melakukan pencarian secara sekuensial (linear search) dari indeks awal array hingga menemukan data dengan NIM yang sama; ketika pertama kali ditemukan, fungsi langsung mengembalikan nilai tersebut. Kedua, fungsi cariNilaiTerbesar juga melakukan iterasi terhadap seluruh data, tetapi kali ini mencari nilai maksimum dari semua data yang memiliki NIM yang sama, dengan membandingkan setiap nilai dan menyimpan nilai terbesar sementara.

Setelah kedua fungsi selesai dijalankan, hasilnya ditampilkan ke layar. Jika NIM tidak ditemukan, program dapat mengembalikan nilai default (misalnya -1), namun jika ditemukan, program akan mencetak nilai pertama dan nilai terbesar dari mahasiswa dengan NIM tersebut. Secara teknis, alur program berjalan dari input jumlah data → pengisian array → pencarian linear berdasarkan NIM → perhitungan nilai maksimum → output hasil.

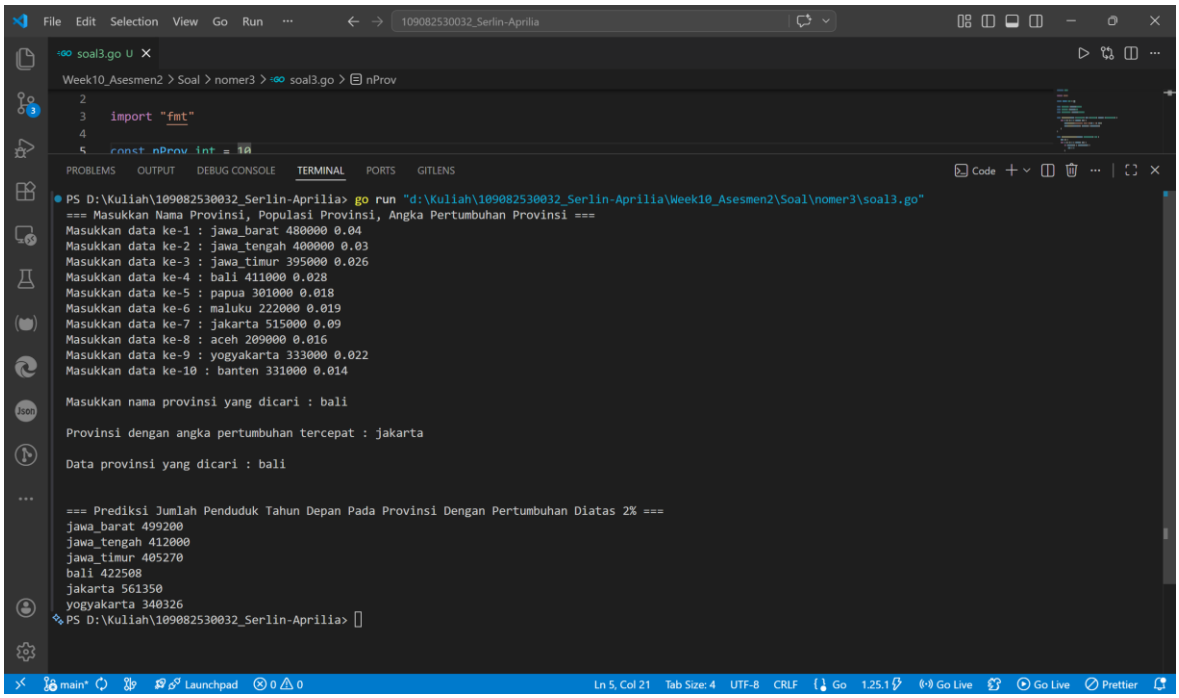
3. Sourec Code Jawaban :

```go
package main

import "fmt"

const nProv int = 10

type namaProv [nProv]string
type popProv [nProv]int
type tumbuhProv [nProv]float64

func inputData(prov *namaProv, pop *popProv, tumbuh *tumbuhProv) {
    for i := 0; i < nProv; i++ {
        fmt.Printf("Masukkan data ke-%d : ", i+1)
        fmt.Scan(&prov[i], &pop[i], &tumbuh[i])
    }
```

```go
}

func provinsiTercepat(tumbuh tumbuhProv) int {
    idx := 0
    for i := 1; i < nProv; i++ {
        if tumbuh[i] > tumbuh[idx] {
            idx = i
        }
    }
    return idx
}

func indeksProvinsi(prov namaProv, nama string) int {
    for i := 0; i < nProv; i++ {
        if prov[i] == nama {
            return i
        }
    }
    return -1
}

func prediksi(prov namaProv, pop popProv, tumbuh tumbuhProv) {
    fmt.Println("\n=== Prediksi Jumlah Penduduk Tahun Depan Pada Provinsi
Dengan Pertumbuhan Diatas 2% ===")
    for i := 0; i < nProv; i++ {
        if tumbuh[i] > 0.02 {
            prediksi := int(float64(pop[i]) * (1 + tumbuh[i]))
            fmt.Println(prov[i], prediksi)
        }
    }
}

func main() {
    var prov namaProv
    var pop popProv
    var tumbuh tumbuhProv
    var cari string

    fmt.Println("=== Masukkan Nama Provinsi, Populasi Provinsi, Angka
Pertumbuhan Provinsi ===")
    inputData(&prov, &pop, &tumbuh)

    fmt.Print("\nMasukkan nama provinsi yang dicari : ")
    fmt.Scan(&cari)
```

```go
    idx := provinsiTercepat(tumbuh)
    fmt.Println("\nProvinsi dengan angka pertumbuhan tercepat :",
prov[idx])

    idxCari := indeksProvinsi(prov, cari)
    fmt.Println("\nData provinsi yang dicari :", cari)

    if idxCari != -1 {
        fmt.Println()
        prediksi(prov, pop, tumbuh)
    }
}
```

Screenshot Output :



Penjelasan Singkat Cara Kerja Program :

Program dimulai dari main dengan mendeklarasikan tiga array statis bertipe namaProv, popProv, dan tumbuhProv yang masing-masing menyimpan nama provinsi, jumlah populasi, dan angka pertumbuhan. Konstanta nProv bernilai 10 digunakan sebagai batas maksimum data sehingga seluruh proses iterasi selalu berjalan dari indeks 0 hingga 9. Selanjutnya, fungsi inputData dipanggil dengan parameter berupa pointer ke ketiga array tersebut. Di dalam fungsi ini dilakukan perulangan sebanyak nProv kali, dan pada setiap iterasi program membaca tiga input sekaligus

menggunakan fmt.Scan, yaitu nama provinsi, populasi, dan angka pertumbuhan, lalu langsung menyimpannya ke indeks array yang sesuai.

Setelah data terisi, program meminta input nama provinsi yang ingin dicari dan menyimpannya ke variabel cari. Kemudian fungsi provinsiTercepat dipanggil dengan parameter array tumbuh. Fungsi ini melakukan pencarian indeks dengan nilai pertumbuhan terbesar menggunakan metode perbandingan linear, dimulai dari indeks 0 sebagai acuan awal, lalu membandingkan setiap elemen berikutnya hingga ditemukan nilai maksimum. Indeks dari nilai maksimum tersebut dikembalikan ke main, lalu digunakan untuk menampilkan nama provinsi dengan pertumbuhan tercepat.

Berikutnya, fungsi indeksProvinsi digunakan untuk mencari posisi indeks dari provinsi yang diinput user. Fungsi ini melakukan pencarian linear terhadap array prov dengan membandingkan string satu per satu. Jika ditemukan, indeks dikembalikan, jika tidak ditemukan maka mengembalikan -1. Setelah itu program tetap menampilkan nama provinsi yang dicari sebagai bagian dari output.

Terakhir, jika indeks valid (tidak -1), fungsi prediksi dipanggil untuk menghitung prediksi jumlah penduduk tahun depan. Di dalam fungsi ini dilakukan iterasi seluruh data, lalu hanya provinsi dengan nilai pertumbuhan lebih dari 0.02 (2%) yang diproses. Perhitungan prediksi dilakukan dengan rumus (1 + pertumbuhan) * populasi, kemudian dikonversi ke integer. Hasilnya ditampilkan dalam bentuk pasangan nama provinsi dan jumlah prediksi penduduk. Secara keseluruhan, alur program berjalan dari input data → penyimpanan array → pencarian maksimum → pencarian indeks berdasarkan nama → seleksi data berdasarkan kondisi → perhitungan → output.